

URL Shortener Web Application (Basic)

1. Introduction

A URL Shortener is a web application that converts a long URL into a much shorter and manageable version. When a user clicks on the shortened URL, they are automatically redirected to the original long URL. URL shorteners are widely used by companies and individuals to improve user experience, track link usage, and make links visually cleaner.

The objective of this project is to design and implement a basic URL Shortener web application using Flask as the backend framework and SQLAlchemy ORM for database interaction. The application allows users to shorten URLs, store them in a database, and view previously shortened URLs. Additionally, the application validates user input to ensure only correct URLs are processed.

2. Technology Stack

This project uses the following technologies:

Flask: Flask is a lightweight and flexible Python web framework. It is easy to learn and well-suited for small to medium-sized web applications. Flask handles routing, request processing, and rendering HTML templates in this project. Its simplicity makes it ideal for backend development learning purposes.

SQLAlchemy ORM: SQLAlchemy is an Object Relational Mapping (ORM) tool that allows developers to interact with databases using Python objects instead of raw SQL queries. In this project, SQLAlchemy is used to define database models, insert data, and retrieve records efficiently. It improves code readability and reduces database-related errors.

SQLite: SQLite is a lightweight, file-based relational database system. It does not require a separate server and is suitable for small-scale applications. For this project, SQLite is used to store original URLs and their corresponding shortened versions.

Bootstrap: Bootstrap is a front-end CSS framework used to design responsive and visually appealing web pages. It helps create a clean user interface without writing extensive custom CSS.

3. System Architecture

The URL Shortener application follows a client-server architecture consisting of three main components:

Frontend

The frontend is built using HTML, CSS, and Bootstrap. It consists of two main pages:

- Home Page: Allows users to enter a long URL and receive a shortened URL.
- History Page: Displays all previously shortened URLs along with their original URLs.

User actions such as submitting a URL are handled through HTML forms.

Backend

The backend is implemented using Flask. It handles:

- Processing user requests
- Validating URLs
- Generating unique short codes
- Saving data into the database
- Redirecting shortened URLs to their original destinations

Flask routes act as the bridge between the frontend and the database.

Database

The database stores all URL-related data. SQLAlchemy ORM is used to interact with the SQLite database. The backend communicates with the database to store and retrieve URL information.

4. URL Validation

Validating user input is a critical part of any web application. In this project, URL validation is performed to ensure that users enter valid URLs before shortening them.

The Python validators library is used to check whether the entered string follows the correct URL format. If the URL is invalid, the application displays an error message and does not store the data in the database. This prevents incorrect or malicious input and improves the reliability of the application.

Example validation logic:

- If the URL is valid → proceed with shortening
- If the URL is invalid → show an error message

5. Database Design

The database design is simple and efficient. A single table is used to store URLs.

Column Name	Data Type	Description
id	Integer	Primary key
original_url	String	Stores the original long URL
short_code	String	Stores the generated short code

ORM Usage: SQLAlchemy ORM maps this table to a Python class. Each row in the table is treated as an object, allowing easy insertion and querying. ORM simplifies database operations and avoids writing raw SQL queries.

6. Challenges Faced

URL Validation: One of the challenges was ensuring that users enter only valid URLs. This was resolved by integrating the validators library to automatically check the correctness of URLs.

Unique Short Code Generation: Generating a unique short code for each URL was another challenge. This was solved by randomly generating alphanumeric strings and ensuring they are unique before storing them in the database.

Database Integration: Connecting Flask with SQLAlchemy and managing database sessions required careful configuration. This was handled by properly initializing the ORM and creating tables within the application context.

7. Conclusion

This project successfully demonstrates the implementation of a basic URL Shortener web application using Flask, SQLAlchemy ORM, and SQLite. Through this project, important backend development concepts such as routing, form handling, database integration, URL validation, and frontend-backend communication were learned.

Future improvements can include:

- User authentication
- Click analytics for shortened URLs
- Expiry time for links
- Custom short URLs
- Deployment on cloud platforms

Overall, this project provided valuable hands-on experience in backend web development and strengthened understanding of Flask-based applications.